

Characterizing Schedules Based on Recoverability

Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

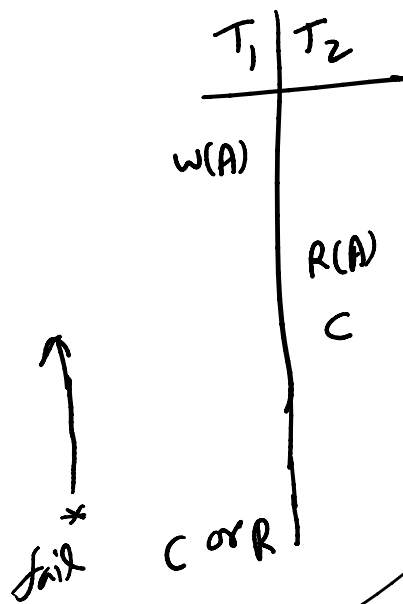
- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

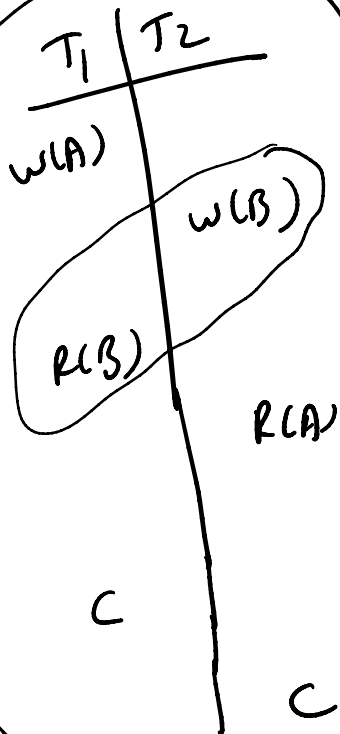
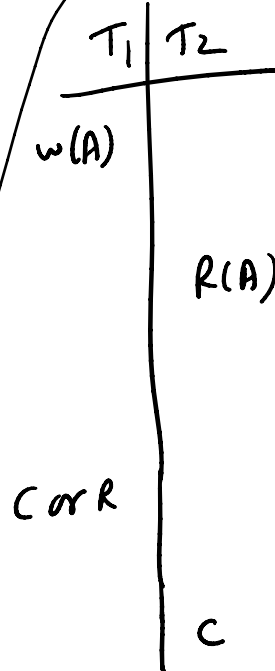
- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

First, we would like to ensure that, once a transaction T is committed, it should *never* be necessary to roll back T . This ensures that the durability property of transactions is not violated (see Section 20.3). The schedules that theoretically meet this criterion are called *recoverable schedules*. A schedule where a committed transaction may have to be rolled back during recovery is called **nonrecoverable** and hence should not be permitted by the DBMS. The condition for a **recoverable schedule** is as follows: A schedule S is recoverable if no transaction T in S commits until all transactions T' that have written some item X that T reads have committed. A transaction T **reads** from transaction T' in a schedule S if some item X is first written by T' and later read by T . In addition, T' should not have been aborted before T reads item X , and there should be no transactions that write X after T' writes it and before T reads it (unless those transactions, if any, have aborted before T reads X).

Nonrecoverable schedule



recoverable
schedule



not recoverable

Recoverable schedule

T_1	T_2
$w(A)$	
	$R(A)$
c/R	
	c

- ① wR
- ② Rw
- ③ ww

T_1	T_2
	$R(A)$
$w(A)$	
	$R(A)$
	$w(A)$
c/R	
	c

T_1	T_2
$R(A)$	
$w(A)$	
	$R(A)$
	$R(B)$
$R(B)$	
$w(B)$	
c	
	c

T_1
 $A \xrightarrow{R_B 500} B$
 $T_2: \text{Sum } A+B$

recov
 But
 inconsistent

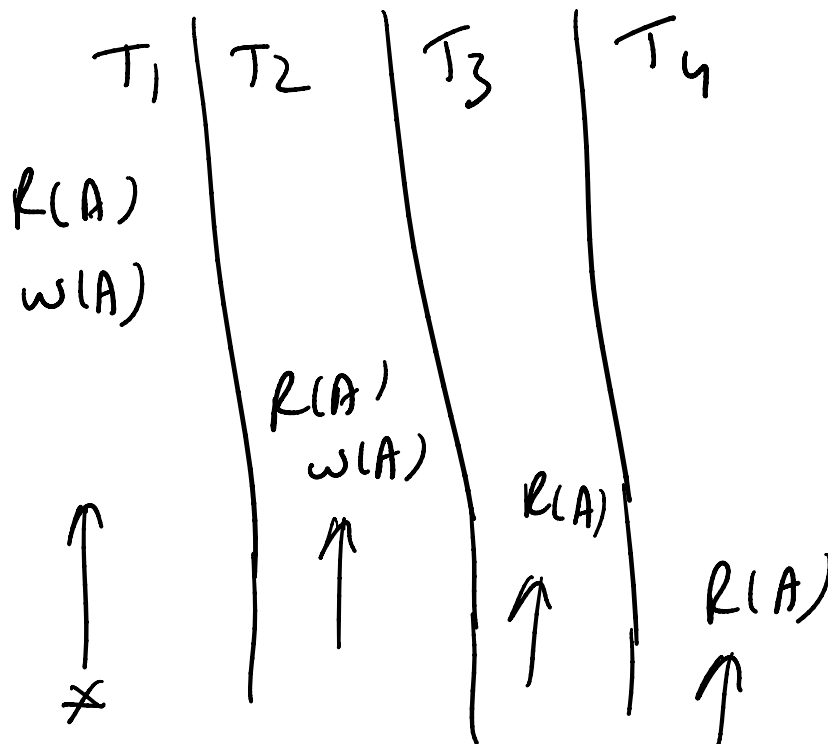
Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	
abort		read (A)

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work

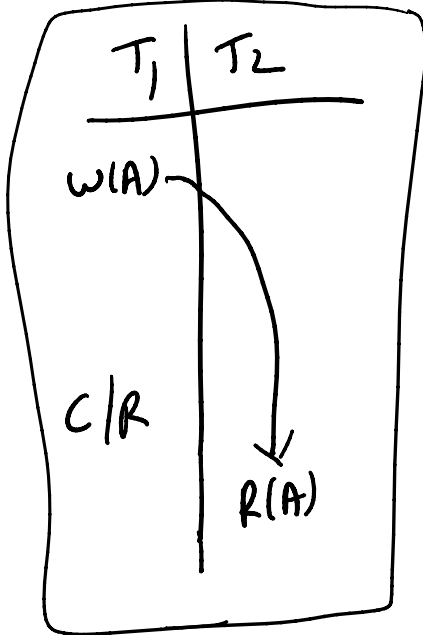


Cascadeless Schedules

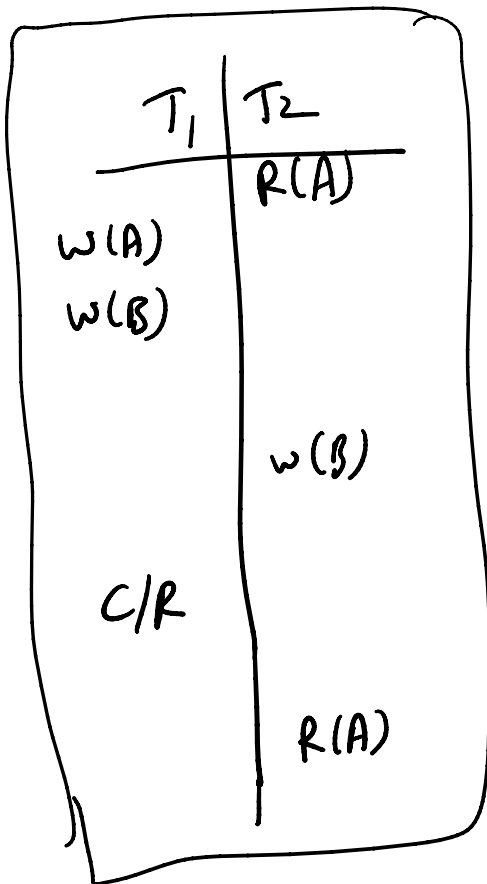
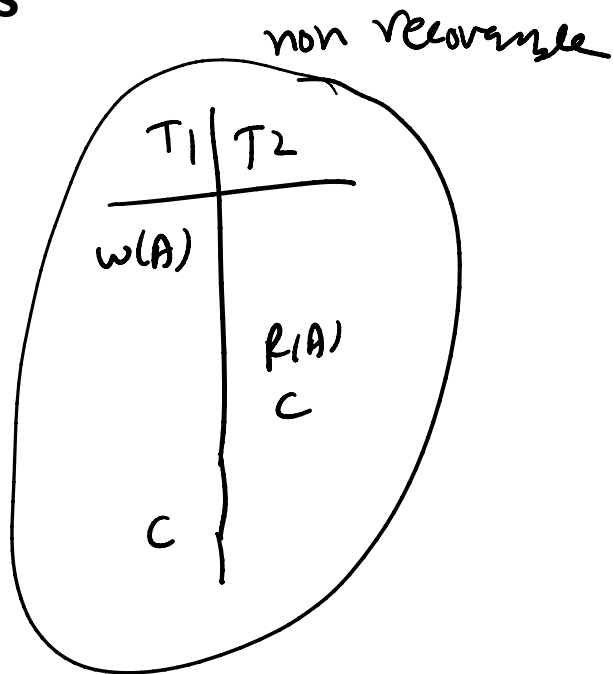
- **Cascadeless schedules** — cascading rollbacks cannot occur;
 - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless

Because cascading rollback can be time-consuming—since numerous transactions can be rolled back (see Chapter 22)—it is important to characterize the schedules where this phenomenon is guaranteed not to occur. A schedule is said to be **cascadeless**, or to **avoid cascading rollback**, if every transaction in the schedule reads only items that were written by committed transactions. In this case, all items read will not be discarded because the transactions that wrote them have committed, so no cascading rollback will occur.

Cascadeless Schedules



recoverable



cascadeless
recoverable

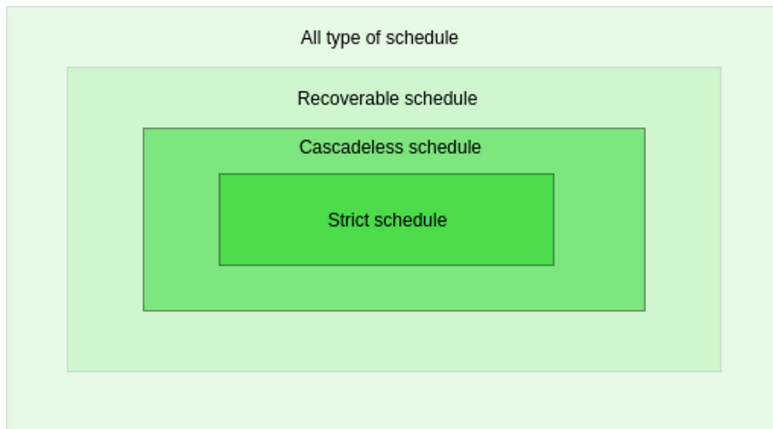
(1) ww

(2) rw

Strict schedule

Finally, there is a third, more restrictive type of schedule, called a **strict schedule**, in which transactions can neither read nor write an item X until the last transaction that wrote X has committed (or aborted).

It is important to note that any strict schedule is also cascadeless, and any cascadeless schedule is also recoverable. Suppose we have i transactions $T_1, T_2, \dots, T_i$, and their number of operations are $n_1, n_2, \dots, n_i$, respectively. If we make a set of *all possible schedules* of these transactions, we can divide the schedules into two disjoint subsets: recoverable and nonrecoverable. The cascadeless schedules will be a subset of the recoverable schedules, and the strict schedules will be a subset of the cascadeless schedules. Thus, all strict schedules are cascadeless, and all cascadeless schedules are recoverable.



T_1	T_2
$w(A)$	
C/R	
	$R(A) / w(A)$

T_1	T_2
	$R(A)$
$w(A)$	
$C \text{ or } R$	
	$R(A) / w(A)$

(D) RW

T_1	T_2
$w(A)$	
	$R(A)$ C
C	

non recoverable

T_1	T_2
$w(A)$	
	$R(A)$
C/R	C

recoverable

T_1	T_2
$w(A)$	
C/R	$R(A)$

Cascadeless

T_1	T_2
$w(A)$	
C/R	$R(A)/w(A)$

Strict schedule

Q $r_1(x) r_3(z) r_1(z) r_3(x) r_3(y) w_1(x) w_3(y) r_2(y) w_2(z)$

$w_2(y) C_1 C_2 C_3$

Nonrecoverable schedule

Q $r_1(x) r_2(z) r_3(x) r_1(z) r_2(y) r_3(y) w_1(x) C_1 w_2(z) w_3(y)$

$w_2(y) C_3 C_2$

not strict

Cascadeless but not Strict schedule

T_1	T_2	T_3
$r(x)$		$r(z)$
$r(z)$		$r(x)$ $r(y)$
$w(x)$		$w(y)$
	$r(y)$	
	$w(z)$	
	$w(y)$	
C_1	C_2	C_3

non
relov

Q $r_3(x) r_1(x) w_3(x) C_3 r_2(x) w_1(y) C_1 r_2(y) w_2(x) C_2$

Strict schedule

Q $r_3(x) r_1(x) w_3(x) r_2(x) w_1(y) r_2(y) w_2(x) C_3 C_1 C_2$

Recoverable but not Cascadeless